Криптографія
Комп’ютерний практикум №3

Криптоаналіз афінної біграмної підстановки

Виконали:
студенти групи ФБ-41
Александрова М. та Проценко Д.

Мета: Набуття навичок частотного аналізу на прикладі розкриття моноалфавітної підстановки; опанування прийомами роботи в модулярній арифметиці.

1. Реалізувати підпрограми із необхідними математичними операціями: обчисленням оберненого елементу за модулем із використанням розширеного алгоритму Евкліда, розв'язуванням лінійних порівнянь.
При розв'язуванні порівнянь потрібно коректно обробляти випадок із декількома розв'язками, повертаючи їх усі.

У нашій роботі ми використовуємо алфавіт з 31 літери: «ё» замінено на «е», «ъ» на «ь»

```
alphabet = "абвгдежзийклмнопрстуфхцчшщьыэюя"
m = len(alphabet)  # 31
n = m * m          # 961 = m^2
```

У афінній біграмній підстановці відкритий текст розбивається на біграми. Біграма (x_1, x_2) відповідає числу $X = x_1 \cdot m + x_2$ від 0 до $m^2 - 1 = 960$, тому всі обчислення шифру виконуються за модулем $n = m^2 = 961$.

Реалізація розширеного алгоритму Евкліда:

```
# розширений алгоритм Евкліда: повертає d, u, v, де d = gcd(a, b) = u*a + v*b
def egcd(a, b):
    r0, r1 = a, b
    u0, u1 = 1, 0
    v0, v1 = 0, 1
    while r1 != 0:
        q = r0 // r1
        r0, r1 = r1, r0 - q * r1
        u0, u1 = u1, u0 - q * u1
        v0, v1 = v1, v0 - q * v1
    return r0, u0, v0
```

Окрім пошуку найбільшого спільного дільника, ця функція також знаходить такі цілі числа u та v, за яких виконується рівність $d=u \cdot a+v \cdot b$, де d=gcd(a,b), а u - обернений елемент до a за модулем n.
Задаються значення r0 = a, r1 =b:

на нульовому кроці: $r0=1 \cdot a+0 \cdot b$ (тобто u0=1,v0=0)
для r1: $r1=0 \cdot a+1 \cdot b$ (тобто u1=0,v1=1),
отже, початкові значення u0, u1, v0, v1 також задаються

Далі виконується цикл ділення з остачею до того моменту, доки щалишок не стане = 0 (що означає, що ми знайшли НСД). Остання ненульова остача і буде НСД.

```
# обернений елемент a^(-1) за модулем mod (None, якщо gcd(a, mod) != 1)
def inverse(a, mod):
    d, u, v = egcd(a % mod, mod)
    if d != 1:
        return None
    return u % mod
```

a має обернений елемент за модулем mod, якщо НСД(a, mod) = 1, тобто якщо ці числа взаємнопрості (тому ми додаємо перевірку через if).
За допомогою розширеного алгоритма Евкліда ми знаходимо d, u та v, де u - обернений елемент до a за модулем mod, Функція inverse() викликає egcd: якщо d ≠ 1, повертається None, в іншому випадку - повертається значення u за модулем mod, щоб воно не було від'ємне

Перевірка:

```
1. Обчислення оберненого елемента

a = 12
mod = 961
a^(-1) = 881 mod 961
Перевірка: 12 * 881 mod 961 = 1
```

```python
# розв'язування ax ≡ b (mod mod), повертає список усіх розв'язків
def solve_linear(a, b, mod):
    a = a % mod
    b = b % mod
    d = egcd(a, mod)[0] # d = gcd(a, mod)

    if b % d != 0: # якщо b не ділиться на d, розв'язків нема
        return []

    a1 = a // d
    b1 = b // d
    n1 = mod // d
    x0 = (b1 * inverse(a1, n1)) % n1 # єдиний розв'язок a1*x = b1 (mod n1)

    solutions = []
    for k in range(d):
        solutions.append(x0 + k * n1)
    return solutions
```

solve_linear(): функція розв'язує ax ≡ b (mod mod) і повертає список усіх розв'язків.
d = gcd(a, mod).
Перевірка через if: лінійне порівняння має розв'язки тільки тоді, коли b ділиться на d без остачі. Якщо не ділиться, повертається пустий список.
Розглядаються ще два випадки:

d = 1: розв'язок єдиний, $x = a^{-1} \cdot b \pmod{}$.

d > 1 і b ділиться на d: порівняння має рівно d розв'язків. Позначаємо $a_1 = a/d$, $b_1 = b/d$, $n_1 = mod/d$. Тоді $x_0 = b_1 \cdot a_1^{-1} \bmod n_1$ розв'язує $a_1 x \equiv b_1 \pmod{n_1}$, а всі розв'язки мають вигляд $x_0 + k \cdot n_1$, де $k = 0, 1, \ldots, d - 1$.

Перевірка:

```
2. Розв'язування лінійних порівнянь

1) з одним розв'язком:
7x ≡ 4 (mod 31)
Розв'язки: [5]
Перевірка: 7*5 mod 31 = 4

2) з декількома розв'язками:
10x ≡ 20 (mod 25)
Кількість розв'язків: 5
Розв'язки: [2, 7, 12, 17, 22]
Перевірка: 10*2 mod 25 = 20
Перевірка: 10*7 mod 25 = 20
Перевірка: 10*12 mod 25 = 20
Перевірка: 10*17 mod 25 = 20
Перевірка: 10*22 mod 25 = 20

3) без розв'язків:
10x ≡ 12 (mod 25)
Розв'язків немає

4) за модулем m² = 961 (як у шифрі):
31x ≡ 93 (mod 961)
Кількість розв'язків: 31
Перші розв'язки: [3, 34, 65, 96, 127]
```

2. За допомогою програми обчислення частот біграм, яка написана в ході виконання комп'ютерного практикуму №1, знайти 5 найчастіших біграм запропонованого шифртексту (за варіантом).

```python
def top_bigrams(text, k=5):
    bigrams = []
    for i in range(0, len(text) - 1, 2):
        bigrams.append(text[i:i + 2])
    counts = Counter(bigrams)
    return counts.most_common(k), len(bigrams)
```

За допомогою цієї функції шукаємо 5 найчастіших біграм шифртексту варіанта 5: Кількість появ кожної біграми підраховується класом Counter, а метод most_common(k) повертає k найчастіших біграм.

```
                >>> ЗАВДАННЯ 2 <<<

Довжина шифртексту: 5838
Всього біграм: 2919

5 найчастіших біграм:
вн – 51 шт, частота = 0.01747 (1.75 %)
тн – 51 шт, частота = 0.01747 (1.75 %)
дк – 48 шт, частота = 0.01644 (1.64 %)
хщ – 48 шт, частота = 0.01644 (1.64 %)
ун – 43 шт, частота = 0.01473 (1.47 %)
```

Найчастіші біграми: вн, тн, дк, хщ, ун

3. Перебрати можливі варіанти співставлення частих біграм мови та частих біграм шифртексту (розглядаючи пари біграм із п'яти найчастіших). Для кожного співставлення знайти можливі кандидати на ключ (a, b) шляхом розв'язання системи (1).

Шифр афінної біграмної підстановки:

$$Y_i = (aX_i + b) \bmod m^2,$$

(a, b) - ключ, нам потрібно знайти можливі кандидати на ключ, за допомогою якого ми могли б розшифрувати шифртекст.
a та b є числовими значеннями, тому біграми нам потрібно перевести у число, використовуємо наступну формулу:

$$(x_{2i-1}, x_{2i}) \leftrightarrow X_i = x_{2i-1}m + x_{2i}.$$

```python
# переведення біграми у число X = x1*m + x2
def bigram_to_number(bigram):
    x1 = alphabet.index(bigram[0])
    x2 = alphabet.index(bigram[1])
    return x1 * m + x2


# переведення числа в біграму
def number_to_bigram(X):
    x1 = X // m
    x2 = X % m
    return alphabet[x1] + alphabet[x2]
```

Ідея атаки на шифр:
Припускаємо, що біграма X1 перейшла при шифруванні у біграму Y1 , а біграма X2 – у біграму Y2 . Тоді для невідомих параметрів ключа a, b можна скласти систему рівнянь:
Y1 ≡ a·X1 + b (mod 961)
Y2 ≡ a·X2 + b (mod 961)

Звідси маємо рівняння для визначення параметру a:
Y1 − Y2 ≡ a·(X1 − X2) (mod 961), це лінійне порівняння, яке ми знаходимо за допомогою функції find_keys(), яка викликає функцію solve_linear():

```python
def find_keys(X1, Y1, X2, Y2):
    keys = []
    # рівняння (2): Y1 - Y2 = a*(X1 - X2) (mod m^2)
    for a in solve_linear(X1 - X2, Y1 - Y2, n):
        if egcd(a, m)[0] != 1:      # a має бути взаємно простим з m
            continue
        b = (Y1 - a * X1) % n       # b = (Y1 - a*X1) mod m^2
        keys.append((a, b))
    return keys
```

НСД(a, m) має = 1, інакше шифр неможливо буде обернути (не існуватиме a^−1).

Після того як знайдено a, значення b обчислюється з будь-якого рівняння системи, ми використовуємо:
b = (Y1 − a·X1) mod 961

Функція candidates() реалізує алгоритм перебору, поєднуючи мовні біграми з біграмами шифртексту:

```
for i in range(len(lang_top)):
    for j in range(i + 1, len(lang_top)):
        X1 = bigram_to_number(lang_top[i])
        X2 = bigram_to_number(lang_top[j])
```

Це цикл по мовних біграмах, він бере всі можливі унікальні пари з 5 найчастіших біграм мови (ст, но, то, на, ен) — таких пар буде (вибираємо 2 з 5)=10. Цикл i + 1 гарантує, що ми не беремо однакові біграми і не дублюємо перевірки.

```
for p in range(len(cipher_top)):
    for q in range(len(cipher_top)):      # пар
        if p == q:
            continue
        Y1 = bigram_to_number(cipher_top[p])
        Y2 = bigram_to_number(cipher_top[q])
```

Це цикл по біграмах шифртексту. Тут перебираються біграми з топ-5 шифртексту. Оскільки порядок у парах важливий (яка з них відповідає X1, а яка X2), тут перевіряються всі впорядковані пари (крім випадків, коли p==q). Всього виходить 5×4=20 варіантів.

Усього 10 · 20 = 200 зіставлень. Порядок біграм мови не враховується, бо зміна порядку в обох парах одночасно дає ті самі рівняння, тому б повторювалось кожне зіставлення двічі.

Алгоритм, реалізований у функції candidates:
Беремо наступну пару біграм мови (X*, X**) і переводимо її в числа функцією bigram_to_number.
Беремо пару біграм шифртексту (Y*, Y**), так само переводимо в числа.
Викликаємо find_keys(X1, Y1, X2, Y2): розв'язуємо порівняння функцією solve_linear за модулем 961, відкидаємо a з gcd(a, 31) ≠ 1 і для решти обчислюємо b = (Y* − a·X*) mod 961.
Додаємо всі знайдені пари (a, b) до списку, пропускаючи ті, що вже траплялись (множина seen).
Ведемо лічильники: скільки зіставлень перевірено, скільки не дали жодного ключа, скільки дали кілька.

```
                >>> ЗАВДАННЯ 3 <<<

Найчастіші біграми російської мови: ['ст', 'но', 'то', 'на', 'ен']
Найчастіші біграми шифртексту: ['вн', 'тн', 'дк', 'хщ', 'ун']

Перевірено зіставлень: 200
Зіставлень без жодного ключа: 68
Зіставлень з кількома ключами: 6
Унікальних кандидатів (a, b): 307
Перші 10:
a = 923 b = 604 ( ст->вн, но->дк )
a = 183 b = 284 ( ст->вн, но->хщ )
a = 86 b = 790 ( ст->тн, но->дк )
a = 307 b = 470 ( ст->тн, но->хщ )
a = 38 b = 566 ( ст->дк, но->вн )
a = 875 b = 876 ( ст->дк, но->тн )
a = 221 b = 775 ( ст->дк, но->хщ )
a = 627 b = 535 ( ст->дк, но->ун )
a = 778 b = 467 ( ст->хщ, но->вн )
a = 654 b = 777 ( ст->хщ, но->тн )
Усі кандидати записані у candidates.txt
```

В результаті отримуємо файл candidates.txt, який містить усіх кандидатів на ключ.

4. Для кожного кандидата на ключ дешифрувати шифртекст. Якщо шифртекст не є змістовним текстом російською мовою, відкинути цього кандидата.

```python
FORBIDDEN = ["ьь", "ыы", "ыь", "ьы", "йь", "йы",
             "аь", "оь", "еь", "иь", "уь", "эь", "юь", "яь"]

FREQ = {
    'о': .110, 'е': .085, 'а': .080, 'и': .074, 'н': .067, 'т': .063,
    'с': .055, 'р': .047, 'в': .045, 'л': .044, 'к': .035, 'м': .032,
    'д': .030, 'п': .028, 'у': .026, 'я': .020, 'ы': .019, 'ь': .019,
    'г': .017, 'з': .016, 'б': .016, 'ч': .014, 'й': .012, 'х': .009,
    'ж': .007, 'ш': .007, 'ю': .006, 'ц': .004, 'щ': .003, 'э': .003,
    'ф': .002,
}

FORBIDDEN_MAX = 0.002
SCORE_MIN = -3.3


def forbidden_rate(text):
    cnt = sum(1 for i in range(len(text) - 1) if text[i:i + 2] in FORBIDDEN)
    return cnt / max(1, len(text) - 1)


def letter_score(text):
    return sum(math.log(FREQ[c]) for c in text) / len(text)


def is_meaningful(text):
    return forbidden_rate(text) <= FORBIDDEN_MAX and letter_score(text) >= SCORE_MIN
```

реалізацію автоматичного розпізнавача змістовності тексту is meaningful(text), який перевіряє дешифрований кандидат за двома критеріями: спочатку функція forbidden rate рахує частку заборонених біграм (наприклад, «аь», «ыы») і відкидає варіант, якщо вона перевищує 0.2% , а потім функція letter_score обчислює середній логарифмічний бал природності літер $\frac{1}{L}\sum \ln(P(c))$ на основі стандартних частот FREQ і відсіює шум із балом нижче -3.3 (SCORE_MIN), повертаючи True тільки для правильного змістовного тексту

c

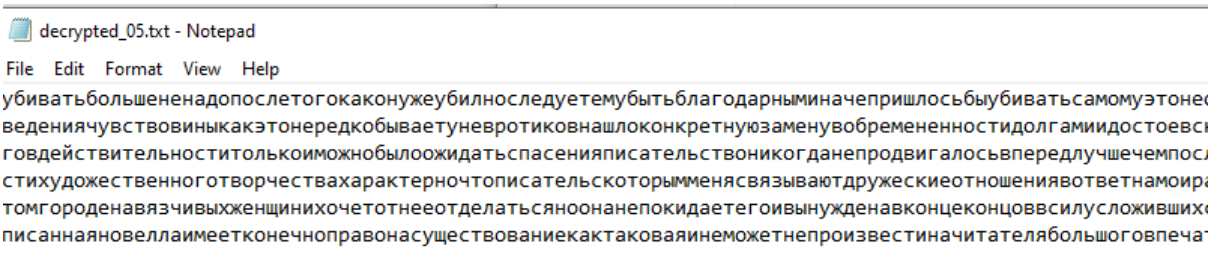
```
    good = []
    for a, b in keys:
        if inverse(a, n) is None:
            continue
        dec = affine_decrypt(cipher, a, b)
        if is_meaningful(dec):
            good.append((letter_score(dec), a, b, dec))
    good.sort(reverse=True)

    print(f"Кандидатів: {len(keys)}, відкинуто: {len(keys) - len(good)}, "
          f"лишилось: {len(good)}")

    # Завдання 5: лишився змістовний текст
    if good:
        sc, a, b, dec = good[0]
        print(f"Ключ: a={a}, b={b}, score={sc:.3f}")
        print(dec[:300])
        with open("decrypted_05.txt", "w", encoding="utf-8") as out:
            out.write(dec)
    else:
        print("Жоден кандидат не пройшов розпізнавач.")
```

Ітеративний перебір реалізовано за допомогою циклу for, який послідовно проганяє всі 307 пар кандидатів (a, b) з файла candidates.txt через функції дешифрування та автоматичного розпізнавання is_meaningful(). У результаті перебору 306 невалідних ключів було відкинуто, і програма автоматично визначила єдиний правильний ключ $a = 654, b = 777$ (з оцінкою $score = -3.067$), який дає повністю змістовний відкритий текст, збережений у файл decrypted_05.txt.

```
Кандидатів: 307, відкинуто: 306, лишилось: 1
Ключ: a=654, b=777, score=-3.067
убиватьбольшененадопослетогокаконужеубилноследуетемубытьблагодарныминачепри
енностьэтойдобротыэтимнеоспариваетсяможетбытьэтовоо
```



Контрольні питання

1. Які статистичні властивості шифртексту були використані під час формування кандидатів на ключ?

Було використано частоти біграм.

Частоти біграм при шифруванні афінною біграмною підстановкою зберігаються, змінюються лише самі біграми, тому що афінний шифр є моноалфавітним на множині біграм: кожна біграма ВТ завжди перетворюється в одну і ту ж біграму ШТ.

Було виділено 5 найчастіших біграм російської мови (lang_top: ст, но, то, на, ен) та 5 найчастіших біграм аналізованого шифртексту (cipher_top). Алгоритм спирається на те, що найбільш часті біграми зашифрованого файлу відповідають найчастішим біграмам природної мови. Зіставляючи їх у парах (X1→Y1 та X2→Y2), ми отримуємо систему рівнянь, за допомогою якої можна обчислити параметри ключа (a,b)

2. Чому не кожне зіставлення частих біграм відкритого тексту та шифртексту приводить до допустимого ключа?

Ми шукаємо кандидатів на ключ через порівняння $Y1 - Y2 \equiv a \cdot (X1 - X2) \pmod{961}$, тому мають задовольнятись дві умови: у порівняння має бути розв'язок a, і це a має бути взаємнопростим з m = 31 (інакше a^{-1} не існує й розшифрування неможливе).

Також причиною може бути статистична похибка: зіставлення Y1 ↔ X1 спирається на припущення, що порядок за частотою у шифртексті збігається з порядком у мові. Частоти найчастіших біграм близькі (у нашому шифртексті 1,25–1,75 %), тому порядок може відрізнятися, і формально допустимий ключ дає при розшифруванні безглуздий текст. Такі ключі відкидає розпізнавач мови.

3. Чи виникали під час криптоаналізу випадки, коли одне зіставлення біграм давало декілька кандидатів на ключ? Як такі кандидати оброблялися?

Так, але лише в тому випадку, коли пара біграм мови це «но» і «то» (коли d = 31), а в біграм шифртексту друга літера однакова. Тоді порівняння має 31 розв'язок, і з одного зіставлення отримується 31 пара (a, b). Усі ці a мають однакову остачу від ділення на 31, тому умова gcd(a, 31) = 1 виконується для всіх одразу. У всіх інших зіставленнях кандидат один або жодного.

Обробка була така:

Функція solve_linear повертає всі розв'язки порівняння.

Для кожного a перевіряється gcd(a, 31) = 1, і обчислюється $b = (Y^* - a \cdot X^*) \bmod 961$.

Усі пари (a, b) заносяться в загальний список разом з описом зіставлення, з якого вони походять (наприклад, «но→дм, то→ке»). Повторювані ключі, що вийшли з різних зіставлень, додаються лише один раз.

Кожен кандидат використовується для розшифрування всього шифртексту, і автоматичний розпізнавач відкидає ті, що дали безглуздий текст. Правильний ключ залишається єдиним, який дає зв'язний російський текст.

4. За якими ознаками серед отриманих кандидатів було визначено правильний ключ?

Правильний ключ було визначено за трьома основними ознаками: повною відсутністю заборонених біграм (частка forbidden_rate = 0%, що менше порогу 0.2%), високим логарифмічним балом частотності літер (letter_score = -3.067, що вище порогу -3.3) та граматичною і семантичною змістовністю отриманого відкритого тексту. Серед усіх

307 кандидатів тільки пара $(a = 654, b = 777)$ задовольнила всім цим умовам, тоді як інші 306 кандидатів були повністю відкинуті.

5. Який критерій автоматичного розпізнавання осмисленого тексту було використано?
Чому було обрано саме його?
Було використано двокритеріальний автоматичний розпізнавач is_meaningful, який поєднує перевірку частки заборонених біграм forbidden_rate (відсіює неможливі поєднання літер) та логарифмічний частотний критерій letter_score за формулою $\frac{1}{L}\sum \ln(P(c))$ (визначає відповідність розподілу букв еталонним частотам мови). Обрано саме цей підхід, оскільки синергія двох фільтрів забезпечує майже 100% точність відсіювання випадкового шуму без потреби використання громіздких словників чи складного морфологічного аналізу, виконуючи перевірку за один швидкий лінійний прохід $O(L)$.

6. Як поводився обраний критерій для правильного відкритого тексту та для результатів
розшифрування неправильними ключами?
Для неправильних ключів (306 із 307) дешифрований текст являв собою випадковий хаотичний шум, де частка заборонених біграм миттєво перевищувала поріг 0.2%, а бал letter_score через часту появу рідкісних літер падав глибоко нижче -3.3 (до -3.6 -4.5), що призводило до їхнього автоматичного відкидання. Для правильного ключа $(a = 654, b = 777)$ заборонені біграми були повністю відсутні (0.0%), а бал letter_score склав -3.067, що дозволило розпізнавачу однозначно виділити цей варіант як єдиний правильний змістовний текст.